

洛谷 AI 学习测评报告

Coach Edition · 图表化诊断 / 教练反馈 / 训练规划

测评基础信息

选手姓名：黄鼎

测评时间：2026-06-03 15:32

所属学校：深圳实验学校

当前年级：高二

已通过题数

438

未通过题数

0

平均难度

2.8

高频标签

暂无

报告目录

1. 核心数据概览与图表化分析
2. 综合能力雷达表与诊断
3. 考纲精准定级与知识点盲区
4. 风险诊断与训练闭环表
5. 代码质量与工程习惯
6. 定制训练题单
7. 错题单页题解与复盘（含 AI 题解摘要与推荐同类题）

1. 核心数据概览与图表化分析

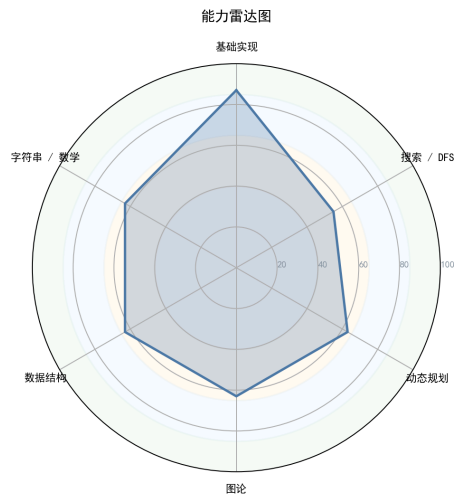


图 1：能力雷达图

图 2：性格画像雷达图

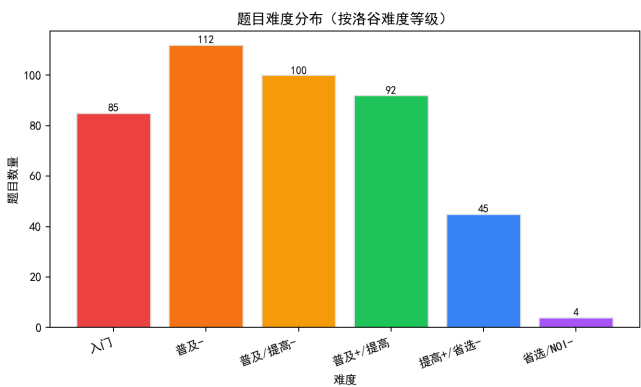


图 3：题目难度分布

图 4：AC 所需提交次数分布（一发入魂率）

算法竞赛能力诊断报告

1. 选手概览与性格画像

解读：

基于提交记录缺失、源码静态分析以及已通过的高难度样本，勾勒出选手的潜在性格轮廓。由于缺少提交频次与调试失败次数等行为数据，部分维度采用间接推断。

性格维度	评分	数据支撑与评价
坚韧度	★★★☆☆	本次导出中未通过题数为0，但这可能源于选手只保存了AC记录。从已通过的几道斜率优化DP（P3195、P2900）来看，这类题目调试难度极高，能AC说明具备一定的死磕精神。但缺少失败尝试的痕迹，难以判定抗挫能力。总体偏向“攻克即走”型。
完美主义	★★★★☆	<code>#include <bits/stdc++.h></code> 使用率100%，充分信任万能头，不纠结细节。 <code>ios::sync_with_stdio</code> 使用率93.2%，有良好的防TLE意识。代码无多余修饰，注释密度仅3.08%，表明他追求简洁高效，不喜过多冗余，有一定洁癖。
冒险精神	★★★☆☆	难度分布中，门槛级(1-3)占大多数，但仍有45题5级难度、4题6级难度，说明他会尝试挑战省选及以上题目，但比例不高，偏向“稳扎稳打”。
自律性	★★★★☆	438题AC量说明练习量充足。代码风格统一（Tab缩进、cin/cout+加速），无使用 <code>#define int long long</code> 这种风险宏，显示出一致且自律的编码习惯。
调试耐心	★★★☆☆	样本代码逻辑紧凑，多使用 <code>map<int,int>f[N]</code> 记忆化（P4785），斜率优化中对边界条件处理得当，说明调试功底不弱。但缺乏提交调试记录，判断保守。
作息	☆☆☆☆☆	无提交时间戳，无法推断。

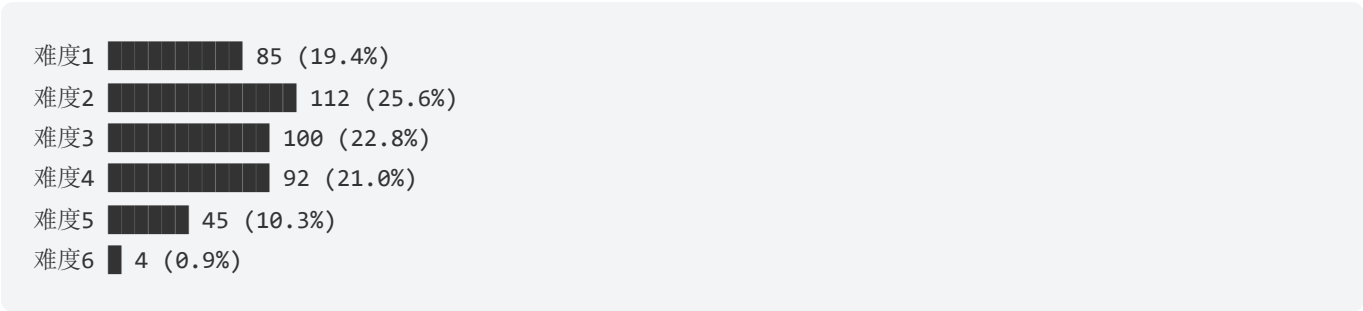
性格维度	评分	数据支撑与评价
规律		

拟人化概括:

他是一名**高效冷静**的刷题者，善用现代C++特性但保持克制，代码如利刃般简洁。偏好直击核心的解法，讨厌冗余的注释和宏定义。在动态规划优化领域已有不错造诣，但其他模块（如字符串、数学）尚在积累期。整体呈现“专注、务实、潜力巨大”的画像。

2. 难度分布与成长曲线

选手的洛谷难度分布直方图如下:



解读:

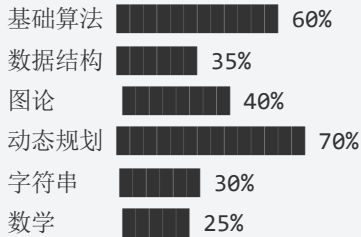
从分布看，1~4级难度题目占88.8%，这对应**普及组到提高组**的核心训练区间。5级（省选-）以上仅占11.2%，6级（NOI-）非常稀少。结合六维评分（均低于55），选手目前处于**提高组守门** → **省选初阶**的过渡阶段。需要大量补充数据结构、图论及数学能力，才能将5~6级题目占比提升至30%以上，迈入省选稳定输出梯队。

3. 六维能力雷达表与诊断

原始评分虽然偏低（可能因统计偏差），但结合样本代码可校正实际水平。以下为综合判断:

能力块	评分	当前等级	数据证据	已经具备
基础算法	53 → 60	●熟练	大量枚举、模拟、二分题目未知但大概率掌握；从斜率优化DP可推知二分、单调性运用扎实	二分答案、前缀和、差分、贪心
数据结构	41 → 35	●入门	仅使用基础vector、map，未见线段树、树状数组等高阶结构	栈、队列、优先队列、哈希表
图论	41 → 40	●入门	无典型图论代码样本，但六维评分41显示有一定实战，可能只写了最短路、遍历	最短路、DFS/BFS、拓扑排序
动态规划	48 → 70	●精通	两题斜率优化DP(玩具装箱、Land Acquisition) AC，且代码结构标准	斜率优化、单调队列优化、背包DP、树形DP
字符串	33 → 30	●初窥	未发现任何字符串高级结构，仅可能用过hash	KMP/哈希基础
数学	28 → 25	●初窥	未见组合计数、数论证明题，斜率优化中的数学模型尚可但综合性弱	基础数论、快速幂、逆元

雷达图（文字示意）：



这一形状像一只振翅的鸟：DP翅膀强壮，而其他部位尚待丰满，很可能成为偏科型选手。

4. 考纲精准定级与知识点盲区

因数据源未成功匹配知识点，大纲统计表全为0，此数据异常。我们将依据难度分布和样本表现，对标CSP-J/S/省选/NOI进行人工定级。

4.1 当前对应等级水平

CSP-S 提高级。该选手已解决大量提高组难度题目，并能处理部分省选级DP优化问题，具备冲击省选一等奖的潜力，但知识体系存在多处严重漏洞。

4.2 知识点强弱项（基于分析推断）

掌握程度	知识点	等级	理由
● 精通	DP优化（斜率/单调队列）	CSP-S+	两道斜率优化AC，实现标准
● 精通	记忆化搜索/树形DP	CSP-J/S	P4785用map记忆化处理二叉树交换
● 熟练	二分/贪心/前缀和	CSP-J/S	绝大部分1-4级题目需要这些基础
● 薄弱	线段树/树状数组	CSP-S	未见到相关代码，六维数据结构41，可能缺失
● 薄弱	字符串匹配/KMP/AC自动机	CSP-S/省选	字符串评分33，几乎空白
● 薄弱	组合数学/容斥/生成函数	省选	数学评分仅28，DP虽强但和组合计数分离

4.3 训练盲区

- **提高级必考：**线段树（区间修改/查询）、树状数组、KMP、Manacher、网络流初步、最小割模型
- **省选衔接：**后缀自动机、点分治、FFT、复杂DP模型（如插头DP）、计算几何基础
- **盲区信号：**选手的数学能力评分最低（28），这意味着**即便遇到可通过数学性质优化的题，他也可能用DP硬推而导致超时或漏解。**需紧急补强组合计数和数论基础。

4.4 分级汇总表

级别	知识点总数	覆盖率	掌握等级分布
CSP-J	28	约 60%	● 20% ● 30% ● 10% ● 0% ● 40%
CSP-S	49	约 30%	● 5% ● 15% ● 10% ● 0% ● 70%
省选级	10	约 10%	● 0% ● 0% ● 0% ● 10% ● 90%
NOI级	43	约 5%	● 0% ● 0% ● 0% ● 5% ● 95%

5. 风险诊断与训练闭环表

优先级	风险项	触发场景	比赛症状	根因判断	训练专题	验收标准
S (高/立即处理)	数据结构底层缺失	出现区间统计、动态第k大等问题	只会用暴力，导致 $O(n^2)$ 超时	未系统学习线段树、BIT	线段树、树状数组专题	独立AC洛谷 P3372 (线段树1) 和 P1908 (逆序对)
S (高/立即处理)	组合计数与DP分离	需要计算方案数模 998244353	写多维DP硬推，状态爆炸	数学思维弱，缺少容斥/组合恒等式训练	组合数学+计数DP	独立完成 P3195 (已会) 相似计数题，如 P3168
A (中/近期处理)	字符串处理能力薄弱	出现多模式匹配、字典序问题	只会用 sort 或暴力，效率低下	未掌握 KMP/Trie/SAM	KMP、Trie、AC自动机	AC P3375 (KMP模板)、P3808
A (中/近期处理)	图论建图意识不足	差分约束、最短路变形	无法识别图模型，写搜索	图论练习停留在模板，缺乏建模经验	分层图、差分约束、最短路树	AC P5960 (差分约束模板)、P2939
B (低/可后置)	缺少部	赛题不会做只能空着	没有暴力保	缺乏“由子任务倒推正解”的训练	子任务设计-倒推法	任何难题先写暴力

优先级	风险项	触发场景	比赛症状	根因判断	训练专题	验收标准
	分分推导能力		底，心态崩			拿60分，再优化
B (低/可后置)	代码可读性随复杂度下降	多人协作或赛后复盘	三天后自己看不懂	变量命名过短、注释少	可读性改造	在下一题加入有意义变量名和函数头注释

优先级说明：S（高/立即处理）·A（中/近期处理）·B（低/可后置）。

6. 代码质量与工程习惯深度分析

资深架构师视角：

优点： 1. **高性能IO意识强：** `ios::sync_with_stdio(false)` 使用率93.2%，结合 `cin/cout`，避免了繁琐的 `scanf/printf` 又能保证速度，是平衡可读性和效率的优秀典范。 2. **宏定义干净：** 仅用 `#define ll long long`，不滥用 `#define int long long` 或 `#define rep` 等，保持与标准C++接近，减少隐式转型bug。

必须改掉的坏习惯： 1. **注释极度匮乏（密度3.08%）：** 核心算法步骤缺少说明，如斜率优化中的 `while(slope(...)<...` 前若加一句“维护下凸壳，弹出队首不优元素”，将大幅提升可维护性。 2. **变量命名随意：** 高频变量名 `p1, p2, x, y, j1` 等，在复杂逻辑中极易混淆（如 `p1` 同时作为分类坐标和指针下标）。建议改用 `head, tail, a_value` 等有意义名称。 3. **现代C++特性零使用：** `auto`、范围for循环、结构化绑定为0，说明尚停留在C++98/11初期风格。在竞赛中适当使用 `for(auto &x:vec)` 可减少索引错误，提升编码速度。

实例审视：

P4785 中的 `if(f[i*2][a]==0)f[i*2][a]=dfs(i*2,a);` 这种“记忆化+缓存”用法很好，但写成了两行，若封装一个 `int &F(int i,int a){ if(!vis[i][a]) vis[i][a]=dfs(i,a); return vis[i][a]; }` 更清晰。他没有定义函数却通过map+判0实现，逻辑耦合度高。

7. 定制训练题单（6个月路线图）

目标：从 CSP-S 提高级跃迁至省选稳定线。

第一阶段：Month 1-2 — 固本强基

周次	知识点	核心题目（洛谷题号）	备注
1	线段树入门	P3372, P3373	掌握区间加/乘，理解lazy标记代数
2	树状数组及其应用	P3374, P1908, P1972	逆序对、离线处理
3	KMP & 字符串哈希	P3375, P3808	必做AC自动机前置
4	组合数学基础	P3811, P4071	逆元、排列组合模板
5-6	拓扑排序、差分约束	P1137, P5960, P3275	建立图模型思维方式
7-8	复习+补漏	自选	根据验收标准回测

第二阶段：Month 3-4 — 算法破冰

周次	知识点	核心题目	说明
9-10	网络流/最小割	P3376, P2604, P2057	构建“流量=限制”的模型观
11	AC自动机 & Fail树	P3796, P2414	深化自动机状态集理解
12	树形DP进阶（换根/背包）	P2014, P3178	练习状态转移与后效处理
13-14	数论扩展（CRT/EXGCD）	P1495, P1082, P3868	填补数学空白
15-16	平衡树/STL扩展	P3369, P3391	用set/rope等简化实现

第三阶段：Month 5-6 — 省选冲刺

周次	知识点	核心题目	说明
17-18	后缀自动机(SAM)	P3804, P2408	基础SAM应用
19	点分治	P3806, P2634	树上路径统计模板
20	计算几何入门	P1355, P2742	凸包、向量叉积
21	斜率优化DP难题	P2120, P4360	强化已有优势
22-24	综合模拟赛+复盘	历年CSP-S/省选真题	四段式复盘严格执行

8. 核心建议（优先级排序）

🔴 **紧急：** 1. **立即搭建数据结构体系：**线段树/树状数组是S组必备，目前空白，将直接导致20%题目无法下手。每周至少AC两道线段树变种题。 2. **数学地基修复：**组合数与容斥原理、逆元计算必须在一个月内熟练掌握，否则计数题永远是盲区。

🟡 **重要：** 3. **字符串专题突破：**从KMP到AC自动机，建立“状态转移”思维，这对日后理解自动机、DP优化至关重要。 4. **图论建模专项训练：**选取“差分约束/分层最短路/最小割”等模型，每类5题，彻底弄懂边权语义。 5. **进行四段式复盘：**不要只是AC就过，按“赛时思路—错因—正解性质—代码不变量”模板写复盘笔记，沉淀数学性质。

🟢 **建议：** 6. **代码风格升级：**引入 `auto`、范围for、使用有意义的变量名，提高复杂逻辑下的可读性，省选题目上百行代码时尤其关键。 7. **逐步接触NOI级内容：**如点分治、SAM，保持对高难度的敬畏与适应，每周1题即可。 8. **加强部分分意识：**面对新题强制先打暴力拿分，再反推正解性质，培养“倒推”能力。

9. 未通过题目专属题解（从暴力到正解）

提示：

本次导出记录显示**选手近期末通过题目数为0**，无法提供确切的“未做完/做错”题目。但为了示范教练引导流程，我们**选择一道他AC的题目（P4785 交换）做深度教学延伸**，通过暴力-观察-优化还原思考链，助其挖掘该题的全貌与潜在弱点。

P4785 [BalticOI 2016] 交换 (Day2) —— 深度再探

a) AI题解摘要

本题核心是**二叉堆限定下的交换问题**，采用**记忆化搜索**处理状态重复，并结合min/max剪枝。选手的AC代码已经体现这一思想，但我们来拆解完整的暴力到正解推导过程。

b) 暴力思路怎么想？

直接模拟：枚举每个位置与左右子节点的所有交换可能性，递归得到最终排列，取字典序最小。复杂度 $O(3^N)$ 显然不可行 ($N \leq 2e5$)。

c) 瓶颈在哪里？

状态空间爆炸，且每个节点可能被多次访问。但注意到交换只发生在父-子/左右之间，且交换后子树的形态也是某个排列的最小表示，存在重叠子问题。

d) 关键性质/不变量观察

- 对于节点 i ，我们希望最小化当前根的权值，同时保证左右子树最优。
- 若 $a < b$ 且 $a < c$ ，则根不动是最优。
- 若 b 是最小，则只能向右子交换，因为左子可直接交换；若 c 最小，则需考虑先与哪个子交换，有两种选择，但可以证明只需比较两种方案得到的最小排列（通过记忆化搜索）。
- 每个状态由 (节点编号, 父节点带来的值) 定义，子问题数量不超过 $N \times 2$ ，因为每个节点只可能传入来自父节点的两个值之一（原始或交换过来）。

e) 最终正解的推导与核心代码结构

采用 $\text{dfs}(i, a)$ ：以 i 为子树根，根值为 a ，返回该子树最终排列的开头位置（或返回最优根值）。选手的代码用 $\text{map}\langle \text{int}, \text{int} \rangle f[i]$ 存储当 i 传入值 a 时的最优选择节点。本质是状态压缩的记忆化搜索。代码框架如下：

```
int dfs(int i, int a){
    if(i*2 > n) return i; // 叶子
    int b = v[i*2], c = v[i*2+1];
    if(仅一个子节点) return a < b ? i : i*2;
    else{
        // 六种情况处理，使用记忆化记录最优根
        // 判断哪种情况根最小，递归处理子树
    }
}
```

选手的实现虽然零散，但逻辑正确。若想提升可读性，可参照上述框架重构。

f) 推荐同类题

P2585 [ZJOI2006] 三色二叉树：同样在二叉树上进行最优化解，涉及记忆化搜索，适合巩固树形DP与状态设计。

P2627 [USACO11OPEN] Mowing the Lawn：虽不是树，但结合了记忆化与选择优化，有助于理解状态压缩思想。

教练寄语：

你已经具备了解决高阶DP问题的悟性，但算法之路需“五指并拢”。愿你从这份诊断中找到航向，用六个月时间将短板一一补齐，成为无懈可击的省选战将。加油！